

Ответы к вопросам

Системный аналитик · опорные ответы для проверки · 64 вопроса, 6 тем

1. Архитектура IT-системы

1. За что отвечает Frontend, а за что Backend?

Frontend отвечает за отображение (экран, кнопки, поля) и приём нажатий. Backend отвечает за логику, расчёты и работу с данными: проверка пароля, расчёт суммы заказа, баланс.

2. Чем отличается UI/UX-дизайн (макет в Figma) от Frontend?

Макет это статичный чертёж: цифры и текст вписаны вручную, одинаковы у всех и не меняются. Frontend это рабочая реализация, которая подтягивает реальные данные из backend и показывает их живыми, у каждого свои.

3. Что такое база данных (Database)?

Хранилище системы. Backend сам данные не хранит: он берёт их из БД (баланс, список товаров, профиль) и кладёт туда новые.

4. Кто такой клиент и кто такой сервер?

Клиент это тот, кто просит. Сервер это тот, кто обрабатывает запрос и отвечает.

5. Почему один Backend может быть и клиентом, и сервером?

Клиент и сервер это роли в конкретном запросе, а не ярлыки. Для телефона backend это сервер (отвечает), но когда сам идёт за данными к другому backend, в этом запросе он клиент.

6. Что такое API?

Посредник с контрактом, через который клиент обращается к серверу: передаёт запрос и возвращает ответ. Фиксирует, что можно попросить и что придёт.

7. Что такое контракт и что он гарантирует?

Договорённость о составе: что именно можно попросить у сервера и что он вернёт. Благодаря ему команды frontend и backend работают независимо, пока соблюдают контракт.

8. Что такое протокол и какие бывают?

Набор правил, по которым две системы передают запросы и ответы. Распространённые: REST, SOAP, WebSocket, gRPC, GraphQL.

9. Чем Sync отличается от Async?

Sync (звонок) блокирующий: отправил и ждёшь ответ, ничего другого не делаешь. Async (письмо) неблокирующий: отправил и сразу занят другим, ответ придёт позже.

10. По какому правилу выбирают между Sync и Async?

«Могу ли я продолжить, не дожидаясь ответа?» Да → async (push, письмо). Нет → sync (оплата: нельзя показать чек, пока банк не подтвердил).

2. HTTP и HTTPS

1. Разница между «протокол», «HTTP», «API», «REST»?

Протокол = способ доставки данных. HTTP = один конкретный протокол, самый ходовой. API = контракт (что попросить и что придёт). REST = стиль оформления API поверх HTTP. Это разные слои, не синонимы.

2. Что такое HTTP и схема request-response?

Протокол обмена сообщениями: клиент отправляет запрос (Request), сервер обрабатывает и отправляет ответ (Response).

3. Что значит, что HTTP stateless?

HTTP не помнит прошлые запросы. Каждый запрос самодостаточен и содержит всё нужное, чтобы сервер его понял.

4. Кто начинает первым?

Всегда клиент. Сервер сам по себе молчит и только отвечает на запросы, первым не пишет.

5. Чем HTTPS отличается от HTTP?

HTTPS это тот же HTTP, но тело шифруется. По обычному HTTP данные едут открытым текстом, и перехватчик прочитает пароль; по HTTPS по пути виден только нечитаемый шифр.

6. Четыре части HTTP-запроса?

Метод (что делаем), URL (у кого/чего), headers (служебная информация), body (сами данные, не всегда есть).

7. Что делают методы?

`GET` прочитать, `POST` создать, `PUT` заменить целиком, `PATCH` изменить частично, `DELETE` удалить.

8. PUT vs PATCH?

`PUT` заменяет объект целиком (непереданные поля затираются). `PATCH` меняет только присланные поля, остальное не трогает.

9. Что такое endpoint, path, query?

Endpoint это конкретный адрес-«дверь» на сервере. Path говорит, какой ресурс (`/users/5`). Query это уточнения после `?` (`?fields=name`).

10. Path vs query?

Path говорит ЧТО за ресурс, query уточняет КАК его отдать (отфильтровать, отсортировать, сколько штук).

11. Что такое status code и группы?

Трёхзначный код ответа. Группы по первой цифре: 2xx успех, 3xx редирект, 4xx ошибка клиента, 5xx ошибка сервера.

12. Правило «4xx клиент, 5xx сервер»?

4xx значит виноват запрос (прислали кривое, не туда, без прав). 5xx значит сломался сам сервер при обработке. По первой цифре сразу ясно, где искать.

13. 401 vs 403?

401 «я не знаю, кто ты» (не залогинен, лечится входом). **403** «я знаю, кто ты, но тебе сюда нельзя» (не хватает прав).

14. Зачем нужны Content-Type, Authorization, Accept?

Content-Type : в каком формате едет тело. **Authorization** : кто вы и есть ли право (обычно токен). **Accept** : в каком формате хотите получить ответ.

15. Что такое body и у каких методов есть?

Сами данные запроса/ответа. Есть у **POST** , **PUT** , **PATCH** ; у **GET** и **DELETE** обычно нет.

16. Что такое REST и его главная идея?

Стиль оформления API поверх HTTP. Главная идея: всё это ресурс (существительное), а метод это действие над ним (глагол).

17. Что такое идемпотентность?

Операция идемпотентна, если повторить её несколько раз это то же самое, что выполнить один раз; результат от повторов не меняется.

18. Какие методы идемпотентны?

GET , **PUT** , **DELETE** идемпотентны. **POST** нет: два **POST** создадут два объекта (поэтому двойное «Оплатить» опасно).

3. Форматы данных

1. Зачем формат вместо «просто текста»?

Обе стороны должны одинаково понимать структуру: где какое поле и где его границы. Формат задаёт строгие правила записи, чтобы сервер парсил однозначно, а не гадал.

2. Что такое JSON?

Способ записи данных парами «ключ: значение», понятный человеку и машине. Формат номер один для REST API.

3. Типы значений в JSON?

Строка в кавычках, число без кавычек, **true** / **false** , **null** , массив **[]** , вложенный объект **{ }** .

4. Почему ведущий ноль пишут строкой?

Как число ведущий ноль отпадёт (**012345** → **12345**). Поэтому ИИН, индекс, телефон, номер счёта (не участвуют в вычислениях) пишут строкой в кавычках.

5. Что такое XML и чем отличается от JSON?

Формат, где каждое поле оборачивается в парный тег **<тег>значение</тег>** . Многословнее JSON, но строже и умеет проверку структуры по схеме.

6. Когда встречается XML, а не JSON?

В корпоративных и государственных системах, чаще всего в SOAP.

7. Что такое SOAP и почему всегда XML?

Строгий протокол обмена сообщениями, где всё описано заранее контрактом. Работает всегда на XML (жёсткие рельсы против гибкого REST).

8. Что такое XSD и WSDL?

XSD задаёт структуру данных (какие поля, каких типов, обязательны ли). WSDL описывает сам сервис (какие методы, что принимает и отдаёт). XSD это «бланк полей», WSDL это «меню методов».

9. Что такое ШЭП и Smart Bridge?

ШЭП это единый шлюз обмена между госорганами РК, формат SOAP/XML. Smart Bridge это каталог, куда организации публикуют контракты (XSD/WSDL) своих сервисов для подключения.

10. Логин/пароль vs ЭЦП?

Логин/пароль отвечает «кто вы». ЭЦП подписывает каждый запрос, доказывая, что он от этой организации и что его не подменили по дороге (защита целостности).

11. Что такое YAML и где встречается?

Формат для конфигов (настройки приложений, OpenAPI). Данные задаются отступами, без скобок и тегов.

4. API Design

1. Главный RESTful-принцип именования?

Адрес это существительное-ресурс, а действие задаёт метод. Не `/getUser`, а `GET /users/5`.

2. Четыре правила resource naming?

1) Существительные, не глаголы. 2) Множественное число для коллекций. 3) Вложенность для связей, но неглубоко. 4) Фильтры и сортировка через query, а не новыми путями.

3. Почему множественное число?

Единообразие: и список (`/users`), и один объект (`/users/5`) живут на одном ресурсе.

4. Почему фильтры через query?

Чтобы не плодить адреса под каждый случай. Один ресурс `/products`, а query уточняет, какие именно и в каком порядке.

5. Что такое версионирование API?

Новую версию выкладывают на отдельном адресе (`/v2/users`), а старая (`/v1`) работает параллельно, пока клиенты не переедут.

6. Что такое breaking change и когда поднимают версию?

Изменение, после которого старый клиент перестает понимать ответ (убрали/переименовали поле, сменили тип). Только тогда нужна новая версия. Добавить необязательное поле можно без версии.

7. Что такое OpenAPI и Swagger?

OpenAPI это стандарт описания REST API (эндпоинты, методы, параметры, форматы). Swagger это инструменты вокруг него; Swagger UI это живая дока, где запрос можно отправить прямо из браузера.

8. Чем OpenAPI/Swagger лучше текстовой доки?

Это машинночитаемый единый источник правды: все смотрят одни эндпоинты, дока не устаревает, запрос можно тут же выполнить. Текстовая дока устаревает и у каждого своя.

5. Real-time протоколы

1. Почему HTTP не может сам прислать данные?

Работает по циклу запрос-ответ, соединение закрывается после ответа, между запросами связи нет, а первым начинает только клиент. Серверу некуда «постучаться».

2. Что такое Polling и его недостатки?

Клиент постоянно переспрашивает «есть новое?». Недостатки: куча пустых запросов грузят сервер впустую и задержка до следующего интервала опроса.

3. Что такое Long Polling и чем лучше?

Сервер держит запрос открытым и отвечает только когда появились данные. Почти нет пустых ответов, и данные приходят сразу, а не после следующего интервала.

4. Что такое SSE и направление потока?

Клиент один раз открывает соединение, и сервер непрерывно шлёт события. Поток в одну сторону: сервер → клиент, клиент только слушает.

5. Что такое WebSocket и full-duplex?

Одно постоянное соединение, по которому оба могут слать сообщения в любой момент. Full-duplex это двусторонняя связь одновременно (как телефонный разговор).

6. WebSocket vs SSE по направлению?

SSE в одну сторону (сервер → клиент), WebSocket в обе стороны (клиент ⇄ сервер).

7. Как выбирают между SSE и WebSocket?

По направлению: данные текут только от сервера, клиент смотрит → SSE. Обе стороны активно шлют друг другу → WebSocket.

6. Endpoint → UI

1. Что такое endpoint?

Конкретный адрес на сервере, по которому фронтенд забирает или отправляет данные (`/products` , `/videos/42`).

2. В каком виде приходит список?

Сервер отдаёт массив объектов `[ ]` , каждый объект это одна карточка. Длина массива = число карточек: сто объектов → сто карточек.

3. В каком виде приходит одна деталь?

Один объект `{ }`, а не массив. Его поля разъезжаются по местам шаблона страницы.

4. `/products` vs `/products/5` ?

Следствие REST: `/products` это коллекция (список), `/products/5` это один элемент по id.

5. Что такое вложенный ресурс?

Ресурс, принадлежащий другому. Endpoint строится от родителя: отзывы товара это `/products/100221859/reviews`.

6. POST-форма vs GET по направлению?

`GET` это «принеси мне» (данные идут от сервера к UI). `POST` это «на, создай» (данные идут от пользователя к серверу). Метод определяет направление.

7. Где лежат данные формы?

Форма собирает поля в JSON, который едет в теле (body) `POST`-запроса.

8. Почему сервер не возвращает пароль?

Сервер никогда не отдаёт пароль обратно в ответе, это база безопасности.

9. Как поиск и фильтры становятся query?

То, что вводят в строку поиска и выбирают в фильтрах, превращается в query-параметры адреса: `?category=phones&sort=price&q=samsung`. Меняется фильтр → меняется endpoint → приходит другой массив.

10. Как работает пагинация?

Лента грузит по страницам: первый экран `?page=1`, кнопка «ещё» или скролл шлёт `?page=2`, новые карточки дописываются снизу.